

Tor transport stream-framing specification.

The *Tor* transport carries the stream layer over a SINGLE ordered bytestream per connection, using a QUIC-modelled framing layer (“Stream Framing”): STREAM frames append data to streams (FIN flag finishes a direction), opening is implicit in the first frame of a new stream *ID*, and flow control is cumulative — *MAX_DATA* bounds total bytes across all streams, and *MAX_STREAMS_UNI* bounds the total count of opened unidirectional streams, raised by absolute-valued frames. The initial credits come from the connection preamble. This is the least implementation-tested text of the draft: Zebra’s framing layer was removed as unreachable code.

The model: a sender *S* delivers one bulk record of *Total* bytes on one stream, and separately keeps one announcement stream open, finishing and replacing it (as “Announcement Streams” allows and this repository’s Finding 2 recommends), *MaxReplace* times. Frames travel in order; credit grants travel back in order. Three policy switches:

<i>GrantPerRecord</i>	the receiver raises <i>MAX_DATA</i> only after processing a COMPLETE record (record-granularity processing), rather than as bytes arrive.
<i>RaiseOnClose</i>	the receiver raises <i>MAX_STREAMS_UNI</i> as streams finish, <i>QUIC</i> ’s way of maintaining a concurrency limit expressed cumulatively.
<i>SenderReadsConcurrent</i>	the sender reads the preamble’s stream-limit field as a CONCURRENT limit (the preamble’s own word) instead of the cumulative count the framing section defines.

Findings this module reproduces (see `../documents/v2-modeling.md`):

- The draft mandates a minimum *initial_max_stream_data* that covers a maximum record, but sets NO minimum for the connection-level *initial_max_data*. A peer advertising less than one record of connection credit, talking to a record-granularity receiver — both conforming — wedges the connection forever (*framing_wedge.cfg*); the same credits with byte-granularity granting complete fine (*framing_perframe.cfg*).
- The preamble calls the stream-limit fields “concurrent”; the framing section defines them as cumulative. The two readings are both implementable and disagree: a cumulative-reading sender stalls silently when the receiver never raises the limit (*framing_noraise.cfg*), and a concurrent-reading sender is disconnected with *PROTOCOL_ERROR* by a cumulative-enforcing receiver (*framing_concurrent.cfg*) — honest peers reading different sentences of the same section.
- *StrictSingletonSafeHere*: on this transport the announcement-stream replacement race of Finding 1 CANNOT occur — the pipe is ordered, so the old stream’s FIN always precedes the replacement’s first frame. The literal singleton rule is safe on *Tor* and unsafe on *QUIC*, which may explain how it was written.

EXTENDS *Naturals*, *Sequences*, *FiniteSets*

CONSTANT <i>Total</i>	bytes of the bulk record (<i>max</i> record abstracted)
CONSTANT <i>InitMaxData</i>	preamble <i>initial_max_data</i> (connection credit)
CONSTANT <i>InitMaxStreamsUni</i>	preamble <i>initial_max_streams_uni</i>
CONSTANT <i>MaxReplace</i>	announcement stream replacements <i>S</i> performs
CONSTANT <i>GrantPerRecord</i>	see module comment
CONSTANT <i>RaiseOnClose</i>	see module comment
CONSTANT <i>SenderReadsConcurrent</i>	see module comment

BulkSid $\triangleq$ 0

AnnSid(*k*) $\triangleq$ *k* announcement streams are 1 .. 1 + *MaxReplace*, opened in order

VARIABLES

$pipe$,	ordered frames $S \rightarrow R$
$back$,	ordered credit frames $R \rightarrow S$
s_sent ,	bulk bytes S has sent
s_data_lim ,	connection-level credit S may use (last MAX_DATA , cumulative bytes)
s_str_lim ,	cumulative count of uni streams S may open (last $MAX_STREAMS_UNI$)
s_opened ,	uni streams S has opened so far
s_finned ,	uni streams S has finished so far
$r_delivered$,	bulk bytes R has processed
$r_granted$,	connection credit R has advertised
r_str_lim ,	stream-count limit R has advertised
r_opened ,	uni streams R has observed opening
r_ann_open ,	announcement sids R sees open (opened, no FIN yet)
$closed$	"none" "PROTOCOL_ERROR"

$vars \triangleq \langle pipe, back, s_sent, s_data_lim, s_str_lim, s_opened, s_finned, r_delivered, r_granted, r_str_lim, r_opened, r_ann_open, closed \rangle$

$NoCode \triangleq \text{"none"}$

Frames: STREAM open of an announcement stream (length 0, abstracted), its FIN, one bulk data byte, and the two credit-raising frames.

$OpenF(k) \triangleq [t \mapsto \text{"open"}, \quad sid \mapsto k]$
 $FinF(k) \triangleq [t \mapsto \text{"fin"}, \quad sid \mapsto k]$
 $ByteF \triangleq [t \mapsto \text{"byte"}, \quad sid \mapsto BulkSid]$
 $MaxDataF(m) \triangleq [t \mapsto \text{"max_data"}, \quad max \mapsto m]$
 $MaxStreamsF(m) \triangleq [t \mapsto \text{"max_streams"}, \quad max \mapsto m]$

$Init \triangleq$

$\wedge pipe = \langle \rangle \wedge back = \langle \rangle$
 $\wedge s_sent = 0 \wedge s_data_lim = InitMaxData$
 $\wedge s_str_lim = InitMaxStreamsUni \wedge s_opened = 0 \wedge s_finned = 0$
 $\wedge r_delivered = 0 \wedge r_granted = InitMaxData$
 $\wedge r_str_lim = InitMaxStreamsUni \wedge r_opened = 0 \wedge r_ann_open = \{\}$
 $\wedge closed = NoCode$

S sends the next byte of the bulk record, within the connection credit its peer has granted (the per-stream credit never binds: the draft mandates $initial_max_stream_data$ of at least a full record).

$SendBulkByte \triangleq$
 $\wedge closed = NoCode$
 $\wedge s_sent < Total$
 $\wedge s_sent < s_data_lim$
 $\wedge s_sent' = s_sent + 1$
 $\wedge pipe' = Append(pipe, ByteF)$

$\wedge$ UNCHANGED $\langle \text{back}, s_data_lim, s_str_lim, s_opened, s_finned,$
 $r_delivered, r_granted, r_str_lim, r_opened, r_ann_open, closed \rangle$

S opens its announcement stream, or a replacement after finishing the previous one. What “within the stream limit” means is the switch: the cumulative count the framing section defines, or the concurrent count the preamble’s field description suggests.

$OpenAnn \triangleq$
 $\wedge \text{closed} = NoCode$
 $\wedge s_opened = s_finned$ previous announcement stream finished
 $\wedge s_opened \leq MaxReplace$ the initial stream plus $MaxReplace$ replacements
 $\wedge$ IF $SenderReadsConcurrent$
 THEN $(s_opened - s_finned) < s_str_lim$
 ELSE $s_opened < s_str_lim$
 $\wedge s_opened' = s_opened + 1$
 $\wedge \text{pipe}' = Append(\text{pipe}, OpenF(AnnSid(s_opened + 1)))$
 $\wedge$ UNCHANGED $\langle \text{back}, s_sent, s_data_lim, s_str_lim, s_finned,$
 $r_delivered, r_granted, r_str_lim, r_opened, r_ann_open, closed \rangle$

S finishes its open announcement stream, intending to replace it.

$FinAnn \triangleq$
 $\wedge \text{closed} = NoCode$
 $\wedge s_opened = s_finned + 1$
 $\wedge s_finned' = s_finned + 1$
 $\wedge \text{pipe}' = Append(\text{pipe}, FinF(AnnSid(s_opened)))$
 $\wedge$ UNCHANGED $\langle \text{back}, s_sent, s_data_lim, s_str_lim, s_opened,$
 $r_delivered, r_granted, r_str_lim, r_opened, r_ann_open, closed \rangle$

R processes the next frame, in order. A bulk byte may trigger a MAX_DATA raise (immediately, or only once the record is complete, per the switch); an open is checked against the cumulative stream limit and the strict singleton reading; a FIN may trigger a $MAX_STREAMS$ raise.

$RecvFrame \triangleq$
 $\wedge \text{closed} = NoCode$
 $\wedge \text{pipe} \neq \langle \rangle$
 $\wedge$ LET $f \triangleq Head(\text{pipe})$ IN
 $\vee \wedge f.t = \text{“byte”}$
 $\wedge r_delivered' = r_delivered + 1$
 $\wedge$ LET $done \triangleq r_delivered + 1 \geq Total$
 $grant \triangleq$ IF $GrantPerRecord$ THEN $done$ ELSE TRUE
 IN $\wedge r_granted' =$ IF $grant$ THEN $Total$ ELSE $r_granted$
 $\wedge \text{back}' =$ IF $grant \wedge r_granted < Total$
 THEN $Append(\text{back}, MaxDataF(Total))$ ELSE back
 $\wedge \text{pipe}' = Tail(\text{pipe})$
 $\wedge$ UNCHANGED $\langle s_sent, s_data_lim, s_str_lim, s_opened, s_finned,$
 $r_str_lim, r_opened, r_ann_open, closed \rangle$

$$\begin{aligned}
& \vee \wedge f.t = \text{"open"} \\
& \wedge \vee \wedge r_opened + 1 > r_str_lim \\
& \quad \text{the framing section's cumulative limit is exceeded;} \\
& \quad \text{a connection error of type } \textit{PROTOCOL_ERROR} \\
& \wedge closed' = \text{"PROTOCOL_ERROR"} \\
& \wedge \text{UNCHANGED } \langle pipe, back, s_sent, s_data_lim, s_str_lim, \\
& \quad s_opened, s_finned, r_delivered, r_granted, \\
& \quad r_str_lim, r_opened, r_ann_open \rangle \\
& \vee \wedge r_opened + 1 \leq r_str_lim \\
& \wedge r_opened' = r_opened + 1 \\
& \wedge r_ann_open' = r_ann_open \cup \{f.sid\} \\
& \wedge pipe' = Tail(pipe) \\
& \wedge \text{UNCHANGED } \langle back, s_sent, s_data_lim, s_str_lim, s_opened, \\
& \quad s_finned, r_delivered, r_granted, r_str_lim, closed \rangle \\
& \vee \wedge f.t = \text{"fin"} \\
& \wedge r_ann_open' = r_ann_open \setminus \{f.sid\} \\
& \wedge \text{LET } raise \triangleq \textit{RaiseOnClose} \\
& \quad \text{IN } \wedge r_str_lim' = \text{IF } raise \text{ THEN } r_str_lim + 1 \text{ ELSE } r_str_lim \\
& \quad \wedge back' = \text{IF } raise \text{ THEN } Append(back, MaxStreamsF(r_str_lim + 1)) \text{ ELSE } back \\
& \wedge pipe' = Tail(pipe) \\
& \wedge \text{UNCHANGED } \langle s_sent, s_data_lim, s_str_lim, s_opened, s_finned, \\
& \quad r_delivered, r_granted, r_opened, closed \rangle
\end{aligned}$$

S processes the next credit frame; limits never decrease.

$$\begin{aligned}
RecvBack & \triangleq \\
& \wedge closed = NoCode \\
& \wedge back \neq \langle \rangle \\
& \wedge \text{LET } f \triangleq Head(back) \text{ IN} \\
& \quad \wedge s_data_lim' = \text{IF } f.t = \text{"max_data"} \wedge f.max > s_data_lim \text{ THEN } f.max \text{ ELSE } s_data_lim \\
& \quad \wedge s_str_lim' = \text{IF } f.t = \text{"max_streams"} \wedge f.max > s_str_lim \text{ THEN } f.max \text{ ELSE } s_str_lim \\
& \wedge back' = Tail(back) \\
& \wedge \text{UNCHANGED } \langle pipe, s_sent, s_opened, s_finned, \\
& \quad r_delivered, r_granted, r_str_lim, r_opened, r_ann_open, closed \rangle
\end{aligned}$$

$$Next \triangleq SendBulkByte \vee OpenAnn \vee FinAnn \vee RecvFrame \vee RecvBack$$

$$Spec \triangleq Init \wedge \Box [Next]_{vars} \wedge WF_{vars}(Next)$$

Liveness: the bulk record is eventually delivered, and every replacement announcement stream eventually opens.

$$BulkDelivered \triangleq \Diamond(r_delivered = Total)$$

$$ReplacementsComplete \triangleq \Diamond(s_opened = MaxReplace + 1)$$

Safety invariants.

$$\begin{aligned} TypeOK &\triangleq \\ &\wedge \quad s_sent \in 0 \dots Total \wedge r_delivered \in 0 \dots Total \\ &\wedge \quad s_opened \in 0 \dots (MaxReplace + 1) \wedge s_finned \in 0 \dots s_opened \\ &\wedge \quad closed \in \{NoCode, "PROTOCOL_ERROR"\} \end{aligned}$$

Finding 1's race is impossible on this transport: the pipe is ordered, so a replacement's open frame is never observed before the previous stream's FIN — the receiver never sees two announcement streams open.

$$StrictSingletonSafeHere \triangleq Cardinality(r_ann_open) \leq 1$$

No interleaving of these conforming-but-divergent peers should end in a protocol error.

$$NoHonestProtocolError \triangleq closed = NoCode$$
